List-Based Scheduling for High-Level Synthesis

Md Azadul Islam
Hochschule Hamm-Lippstadt University of Applied Science
Lippstadt, Germany
md-azadul.islam@stud.hshl.de

***Abstract*—High-Level Synthesis (HLS) translates an algorithmic description into a register-transfer level (RTL) hardware implementation. One of the central HLS tasks is scheduling, in which operations are assigned to control steps while respecting data dependencies and resource constraints. This paper studies List-Based Scheduling as a practical heuristic for resource-constrained scheduling. A small arithmetic example is first analyzed with the unconstrained ASAP and ALAP schedules to derive operation mobilities and a latency lower bound, then implemented as a C++ list scheduler, and finally realized as two VHDL RTL designs. The experiments compare a design with one arithmetic logic unit (ALU) and one multiplier against a design with two ALUs and one multiplier. The C++ scheduler reports a latency of 4 cycles for the one-ALU design and 3 cycles for the two-ALU design; the mobility analysis shows that the 3-cycle result matches the critical-path lower bound and is therefore optimal for this graph. QuestaSim verification confirms that both RTL designs compute the correct result, $z = 46$, while the two-ALU design finishes earlier. An analytic resource comparison quantifies the additional datapath hardware required by the faster design. The results demonstrate the typical HLS tradeoff between hardware resources and latency.**

***Index Terms*—High-Level Synthesis, List-Based Scheduling, ASAP, ALAP, mobility, RTL, VHDL, resource constraints, latency, C++**

I. Introduction

Modern embedded and digital systems often require a compromise between execution speed, hardware area, and design cost. High-Level Synthesis (HLS) supports this process by translating an algorithmic description, for example written in C or C++, into an RTL-level hardware implementation. Teich and Haubelt describe synthesis as an automated design process and present allocation, scheduling, and binding as fundamental synthesis tasks [1]. De Micheli likewise treats scheduling and binding as the central problems of architectural-level synthesis [2]. Surveys of the field trace how these ideas developed from early behavioral-synthesis research [3] into the commercial and academic HLS tools that are used for FPGA and ASIC design today [7]–[9].

This paper focuses on scheduling. Scheduling decides in which clock cycle, or control step, each operation is executed. If hardware resources are limited, independent operations may not be able to run in the same cycle even though their data dependencies would allow it. List-Based Scheduling addresses this by maintaining a list of ready operations and selecting operations according to a priority rule while checking resource availability [2], [6].

The contribution of this work is an educational implementation study that connects three abstraction levels of the same problem. First, the example data-flow graph is analyzed analytically: the unconstrained ASAP and ALAP schedules are computed, the mobility of every operation is derived, and the critical path is used as a latency lower bound. Second, a C++ program implements the list scheduler and produces schedules for two resource configurations. Third, the same schedules are realized in VHDL as finite-state-machine-plus-datapath (FSMD) designs and verified in QuestaSim. This connects the HLS scheduling concept to concrete RTL behavior and makes the resource–latency tradeoff directly observable in simulation.

The remainder of the paper is organized as follows. Section II reviews the background and related scheduling algorithms. Section III introduces the mathematical model of the example. Section IV derives the ASAP/ALAP bounds and mobilities. Section V presents the list-scheduling algorithm and Section VI its C++ implementation. Section VII describes the VHDL realization and simulation. Section VIII discusses the results, Section IX states limitations and future work, and Section X concludes.

II. Background and Related Work

In HLS, an algorithm is mapped to hardware components such as ALUs, multipliers, registers, multiplexers, and a control unit. Three synthesis terms are especially relevant [1], [2]:

- *Allocation* decides the available hardware resources, such as one ALU or two ALUs.
- *Scheduling* assigns operations to clock cycles or control steps.
- *Binding* maps operations to concrete hardware units.

Teich and Haubelt discuss these synthesis tasks independently of the final implementation type and describe operations such as additions and multiplications as tasks that are mapped to resources such as ALUs and multipliers [1]. This is directly related to the example in this paper, where addition and multiplication operations are scheduled under ALU and multiplier constraints.

A. The Scheduling Landscape

Scheduling algorithms are commonly divided into unconstrained and constrained variants [2]. In the *unconstrained* case, only data dependencies limit the schedule. The *as-soon-as-possible* (ASAP) algorithm schedules each operation in the earliest cycle permitted by its predecessors and yields the minimum-latency schedule; the *as-late-as-possible* (ALAP) algorithm schedules each operation in the latest cycle that still meets a given latency bound. The difference between

the ALAP and ASAP start times of an operation is called its *mobility* (or slack) and quantifies the scheduling freedom of that operation. Operations with zero mobility lie on the critical path.

For *resource-constrained* scheduling, exact and heuristic methods exist. The problem can be formulated as an integer linear program (ILP) whose solution is provably optimal [2], [5]; however, resource-constrained scheduling is intractable in general, so ILP formulations scale poorly with problem size. Hu's algorithm solves a restricted special case (single operation type, unit delays, tree-structured graphs) exactly in polynomial time [2]. For general graphs, heuristics dominate in practice. List scheduling, studied since the early work on multiprocessor schedules [6], processes the operations cycle by cycle using a priority list and is fast and simple. Force-directed scheduling by Paulin and Knight instead balances the expected resource usage across control steps to minimize resources under a latency constraint [4]. De Micheli places list scheduling among the heuristic algorithms for resource-constrained scheduling and notes that mobility-based priorities are a common choice [2].

B. Modern HLS Tools

The algorithmic ideas above underpin modern HLS flows. Surveys by Coussy et al. [7] and Cong et al. [8] describe how commercial tools compile C/C++ specifications into RTL through compilation, allocation, scheduling, and binding steps, and Nane et al. evaluate a broad set of academic and commercial FPGA HLS tools [9]. Although production tools add many refinements (operation chaining, pipelining, memory optimization), the scheduling core of several tools is still a list-based heuristic, which motivates studying it on a small, fully traceable example.

III. Mathematical Model of the Example

The arithmetic expression used in the experiment is

$$z = (a + b)(c + d) + e. \tag{1}$$

It is decomposed into four operations:

$$v_1 = a + b, \tag{2}$$
$$v_2 = c + d, \tag{3}$$
$$v_3 = v_1 \times v_2, \tag{4}$$
$$v_4 = v_3 + e. \tag{5}$$

The operation graph is represented as $G = (V, E)$, where V is the set of operations and E is the set of dependency edges:

$$V = \{v_1, v_2, v_3, v_4\}, \tag{6}$$
$$E = \{(v_1, v_3), (v_2, v_3), (v_3, v_4)\}. \tag{7}$$

The edges mean that v_3 cannot start before both v_1 and v_2 finish, and v_4 cannot start before v_3 finishes. Fig. 1 shows the resulting data-flow graph. If $s(v_i)$ denotes the start cycle of operation v_i and $d(v_i)$ its duration, the dependency constraint is

$$s(v_j) \geq s(v_i) + d(v_i), \quad \forall (v_i, v_j) \in E. \tag{8}$$

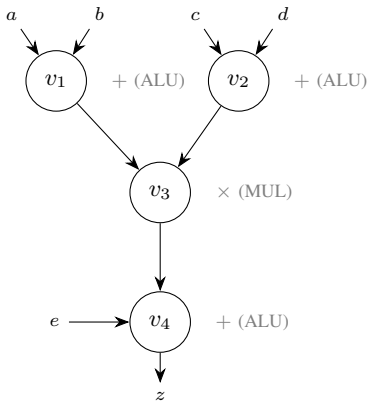

Fig. 1. Data-flow graph of the example $z = (a + b)(c + d) + e$. The critical path $v_1 \to v_3 \to v_4$ (equivalently $v_2 \to v_3 \to v_4$) has length 3.

In this study all operations have unit duration, $d(v_i) = 1$. The resource constraint states that the number of operations of a certain type scheduled in a cycle must not exceed the number of allocated resources of that type. With a_{ALU} allocated ALUs and a_{MUL} multipliers,

$$\left|\{\, v_i : s(v_i) = t \ \wedge\ \mathrm{type}(v_i) = r \,\}\right| \leq a_r \quad \forall t, r. \tag{9}$$

For example, with one ALU, the two additions v_1 and v_2 cannot execute in the same cycle.

IV. Unconstrained Bounds: ASAP, ALAP, and Mobility

Before applying the resource-constrained heuristic, it is instructive to compute the unconstrained bounds, since they establish the best latency any scheduler can achieve for this graph [2].

ASAP schedule. Operations v_1 and v_2 have no predecessors and start in cycle 1. Operation v_3 depends on both and starts in cycle 2. Operation v_4 depends on v_3 and starts in cycle 3. The ASAP latency is therefore

$$L_{\min} = 3 \text{ cycles}, \tag{10}$$

determined by the critical path $v_1 \to v_3 \to v_4$.

ALAP schedule. With the latency bound $\bar{L} = L_{\min} = 3$, the ALAP schedule is computed backwards from the output: v_4 must start in cycle 3, v_3 in cycle 2, and both v_1 and v_2 in cycle 1.

Mobility. The mobility of each operation is $\mu(v_i) = s^{\mathrm{ALAP}}(v_i) - s^{\mathrm{ASAP}}(v_i)$. Table I summarizes the result: for $\bar{L} = 3$ every operation has zero mobility, i.e., *all four operations lie on a critical path* and there is no scheduling freedom. Two consequences follow. First, any schedule achieving 3 cycles must execute v_1 and v_2 in parallel in cycle 1, which requires at least two ALUs; with a single ALU the latency necessarily exceeds the bound and the best possible latency becomes 4 cycles. Second, the 3-cycle schedule found later by the list scheduler for the two-ALU configuration matches $L_{\min}$ and is therefore provably optimal for this graph, even though list scheduling is only a heuristic in general.

TABLE I
ASAP AND ALAP START TIMES AND MOBILITY ($\bar{L} = 3$).

Operation	Type	s^{ASAP}	s^{ALAP}	Mobility μ
$v_1 = a + b$	ALU	1	1	0
$v_2 = c + d$	ALU	1	1	0
$v_3 = v_1 \times v_2$	MUL	2	2	0
$v_4 = v_3 + e$	ALU	3	3	0

V. List-Based Scheduling Algorithm

List-Based Scheduling is a constructive heuristic. It maintains a ready list of operations whose predecessor operations have already been completed. The ready operations are sorted by priority, and the scheduler assigns operations to the current cycle if the required resource is available. De Micheli places list scheduling among heuristic scheduling algorithms for resource-constrained scheduling [2]. Teich and Haubelt also present list scheduling in the context of resource-constrained scheduling [1]. Classic priority functions include mobility (operations with less slack first) and path length to the sink [2], [6].

The simplified algorithm implemented in this work is:

```
Input: operation graph G(V,E), resource limits
Output: start cycle for every operation
while not all operations are scheduled:
  ready_list = operations whose dependencies
    are finished
  sort ready_list by priority
  for each operation in ready_list:
    if required resource is available:
      schedule operation in current cycle
      mark resource as used
  move to next cycle
```

Each operation is visited at most once per cycle and the number of cycles is bounded by the sequential latency, so the run time is polynomial in the number of operations; with a sorted ready list the complexity is $O(|V|^2 \log |V|)$ in the worst case for unit-delay graphs. The method is fast and practical for educational and many engineering cases. However, it is a heuristic and therefore does not guarantee a globally optimal schedule for every possible problem; graphs exist for which a greedy priority choice leads to suboptimal latency [2]. For the graph in Fig. 1 the mobility analysis of Section IV shows that the heuristic result is in fact optimal.

VI. C++ Implementation

The C++ implementation represents each operation using a structure containing its name, type, duration, priority, dependencies, and scheduled start cycle. The function `dependenciesFinished()` checks whether all predecessors of an operation have already completed. In every cycle, the program builds the ready list, sorts it by priority, and schedules operations if the required resource is available.

TABLE II
SCHEDULE WITH ONE ALU AND ONE MULTIPLIER.

Cycle	Scheduled operation
1	$v_1 = a + b$
2	$v_2 = c + d$
3	$v_3 = v_1 \times v_2$
4	$v_4 = v_3 + e$

```
PS C:\Users\User\Downloads\achim\cpp> .\list_scheduler.exe
List-Based Scheduling for High-Level Synthesis
------------------------------------------------

Resource Constraints:
ALU units        : 1
Multiplier units : 1

Scheduled Output:
-----------------
Cycle 1: v1 = a + b [ALU]
Cycle 2: v2 = c + d [ALU]
Cycle 3: v3 = v1 * v2 [MUL]
Cycle 4: v4 = v3 + e [ALU]

Final Operation Start Times:
----------------------------
v1 = a + b starts at cycle 1
v2 = c + d starts at cycle 2
v3 = v1 * v2 starts at cycle 3
v4 = v3 + e starts at cycle 4

Total latency = 4 cycles
```

Fig. 2. C++ List-Based Scheduling output with one ALU and one multiplier. The two addition operations are scheduled in separate cycles, resulting in a latency of 4 cycles.

A. One ALU and One Multiplier

In the first experiment, the resources are one ALU and one multiplier. Since $v_1 = a + b$ and $v_2 = c + d$ both require an ALU, they must be scheduled in separate cycles. The resulting schedule is shown in Table II and the C++ output is shown in Fig. 2. The single-ALU constraint serializes the two additions, pushing v_3 to cycle 3 and v_4 to cycle 4, one cycle beyond the unconstrained bound.

B. Two ALUs and One Multiplier

In the second experiment, two ALUs and one multiplier are available. The two independent additions can therefore be executed in parallel in cycle 1. The resulting schedule is shown in Table III and the C++ output is shown in Fig. 3. The measured latency of 3 cycles equals the ASAP lower bound of Section IV.

VII. VHDL RTL Realization and Simulation

The VHDL implementation realizes the scheduled operations at RTL level and thereby makes the binding step of HLS concrete. Two modules were created: `hls_sched_1alu.vhd` for the 4-cycle schedule and `hls_sched_2alu.vhd` for the 3-cycle schedule.

A. FSMD Architecture

Each module follows the classic finite-state-machine-with-datapath (FSMD) structure that De Micheli describes for control-unit synthesis of scheduled sequencing graphs [2]. The *controller* is a Moore FSM with one state per control step

TABLE III
SCHEDULE WITH TWO ALUS AND ONE MULTIPLIER.

Cycle	Scheduled operation
1	$v_1 = a + b$ and $v_2 = c + d$
2	$v_3 = v_1 \times v_2$
3	$v_4 = v_3 + e$

```
List-Based Scheduling for High-Level Synthesis
------------------------------------------------

Resource Constraints:
ALU units        : 2
Multiplier units : 1

Scheduled Output:
-----------------
Cycle 1: v1 = a + b [ALU], v2 = c + d [ALU]
Cycle 2: v3 = v1 * v2 [MUL]
Cycle 3: v4 = v3 + e [ALU]

Final Operation Start Times:
----------------------------
v1 = a + b starts at cycle 1
v2 = c + d starts at cycle 1
v3 = v1 * v2 starts at cycle 2
v4 = v3 + e starts at cycle 3

Total latency = 3 cycles
```

Fig. 3. C++ List-Based Scheduling output with two ALUs and one multiplier. The two independent addition operations execute in parallel in cycle 1, reducing the latency to 3 cycles.

of the schedule plus an idle and a done state; Fig. 4 shows the controller of the two-ALU design. On `start`, the FSM steps through the control states $S_1 \to S_2 \to S_3$, asserting in each state the register-enable and operand-select signals that correspond to the operations scheduled in that cycle.

The *datapath* contains input registers for $a, \ldots, e$, intermediate registers for v_1, v_2, and v_3, an output register for z, and the allocated functional units. In the one-ALU design a single adder is shared by v_1, v_2, and v_4; multiplexers at the ALU inputs select the operand pair of the current control step, which is exactly the binding of three operations onto one resource. In the two-ALU design a second adder removes the input multiplexers for the parallel additions, so cycle 1 computes v_1 and v_2 simultaneously. Both designs instantiate one combinational multiplier used in a single control step.

B. Testbench and Simulation

A common testbench instantiates both designs, drives them with the same clock and `start` pulse, and applies the input values

$$a = 2, \quad b = 3, \quad c = 4, \quad d = 5, \quad e = 1. \tag{11}$$

The expected result is

$$z = (2 + 3)(4 + 5) + 1 = 5 \cdot 9 + 1 = 46. \tag{12}$$

The QuestaSim waveform in Fig. 5 shows that both designs produce $z = 46$. The signal `done_2alu` becomes high one clock cycle earlier than `done_1alu`, confirming in simulation

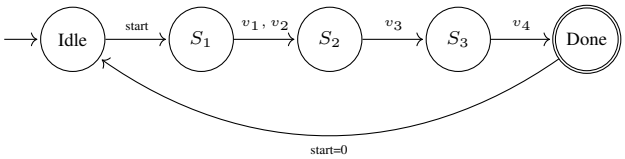

Fig. 4. Controller FSM of the two-ALU design. Each control state corresponds to one control step of the schedule in Table III; the edge labels indicate the operations executed when leaving the state. The one-ALU controller is identical in structure but uses four control states.

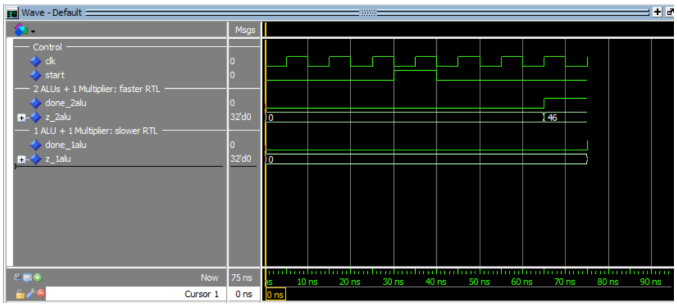

Fig. 5. QuestaSim waveform of the VHDL RTL implementations. Both designs compute the same result $z = 46$. The two-ALU design finishes earlier than the one-ALU design, confirming the resource–latency tradeoff.

that the two-ALU implementation finishes earlier, exactly as predicted by the cycle counts of the C++ scheduler.

VIII. RESULTS AND DISCUSSION

Table IV summarizes the results. The C++ program demonstrates the scheduling decisions, while the VHDL simulation confirms the RTL behavior.

The results show the main tradeoff in resource-constrained HLS scheduling. With one ALU, operations v_1 and v_2 cannot run together, so the schedule requires four cycles. With two ALUs, these independent additions run in parallel, reducing the latency to three cycles—a 25% latency improvement that, by the mobility analysis of Section IV, is the best achievable for this graph.

A. Analytic Area Estimation

The latency gain is paid for in datapath hardware. Table V counts the main datapath components of both designs analytically for a word width of w bits. The two-ALU design adds one w-bit adder but removes the operand multiplexers that the shared ALU requires, since each adder now has dedicated operand sources. For a resource-dominated circuit, area grows roughly with the number of functional units [2], so the second ALU is the dominant cost; the controller shrinks slightly because one control state disappears. This illustrates the area/latency design space that architectural optimization explores [1], [2]: neither design is "better" in isolation—the choice depends on whether the application values the saved cycle more than the extra adder.

TABLE IV
COMPARISON OF RESOURCE CONFIGURATIONS.

Configuration	Behavior	Latency	Output
1 ALU + 1 multiplier	Sequential additions	4 cycles	46
2 ALUs + 1 multiplier	Parallel additions	3 cycles	46

TABLE V
ANALYTIC DATAPATH RESOURCE COMPARISON (w-BIT WORD WIDTH).

Component	1 ALU design	2 ALU design
Adders (w-bit)	1	2
Multipliers ($w \times w$)	1	1
Registers (w-bit)	9	9
2:1 operand multiplexers	≥ 4	≤ 2
FSM control states	6	5

B. Interpretation in the HLS Design Flow

The experiment reproduces, on a minimal example, the loop that HLS tools execute automatically: allocate resources, schedule under the allocation, bind operations to units, and generate an FSMD [7], [8]. Changing a single allocation parameter (the number of ALUs) changed the schedule, the controller, and the datapath structure, while functional correctness was preserved—the essential property that makes HLS-based design-space exploration practical.

IX. LIMITATIONS AND FUTURE WORK

The implementation is intentionally small and educational. It does not include register allocation optimization, memory access scheduling, multiplexer sharing, power estimation, or automatic priority computation; the priority of each operation is assigned manually rather than derived from mobility or path length. All operations complete in one cycle, so multi-cycle and pipelined functional units, operation chaining (executing dependent operations in the same cycle when the clock period allows), and pipelining across loop iterations are not covered [2]. Larger designs may require more advanced heuristics, force-directed scheduling [4], or exact ILP methods [5].

Future work could extend the C++ scheduler to compute ASAP/ALAP mobilities automatically and use them as the priority function, add synthesis of the two VHDL designs to a concrete FPGA target to replace the analytic area estimate of Table V with measured LUT and register counts, and compare the heuristic schedules against an ILP-optimal reference on larger randomly generated graphs. Nevertheless, the example demonstrates the essential idea of List-Based Scheduling and its connection to RTL implementation.

X. CONCLUSION

This paper presented List-Based Scheduling for High-Level Synthesis using a simple arithmetic data-flow example. An ASAP/ALAP mobility analysis established a latency lower bound of three cycles and showed that every operation of the example lies on the critical path. A C++ implementation generated schedules under two resource configurations, and VHDL RTL simulations verified the resulting hardware behavior. Increasing the available ALUs from one to two reduced latency from 4 cycles to 3 cycles—matching the theoretical bound—while both designs produced the correct output, $z = 46$. An analytic resource comparison quantified the hardware cost of the faster design. The experiment demonstrates how List-Based Scheduling exploits parallelism when resources are available and highlights the HLS tradeoff between latency and hardware cost.

AI USAGE STATEMENT

AI assistance was used to organize the explanation, prepare the LaTeX draft, and improve wording. The C++ outputs and VHDL waveform included in this paper were generated from the author's implementation and simulation runs. The final technical content should be checked against the cited references and submitted according to the course rules.

REFERENCES

[1] J. Teich and C. Haubelt, *Digitale Hardware/Software-Systeme: Synthese und Optimierung*, 2nd ed. Berlin, Germany: Springer, 2007.

[2] G. De Micheli, *Synthesis and Optimization of Digital Circuits*. New York, NY, USA: McGraw-Hill, 1994.

[3] M. C. McFarland, A. C. Parker, and R. Camposano, "The high-level synthesis of digital systems," *Proceedings of the IEEE*, vol. 78, no. 2, pp. 301–318, Feb. 1990.

[4] P. G. Paulin and J. P. Knight, "Force-directed scheduling for the behavioral synthesis of ASICs," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 8, no. 6, pp. 661–679, Jun. 1989.

[5] C.-T. Hwang, J.-H. Lee, and Y.-C. Hsu, "A formal approach to the scheduling problem in high level synthesis," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 10, no. 4, pp. 464–475, Apr. 1991.

[6] T. L. Adam, K. M. Chandy, and J. R. Dickson, "A comparison of list schedules for parallel processing systems," *Communications of the ACM*, vol. 17, no. 12, pp. 685–690, Dec. 1974.

[7] P. Coussy, D. D. Gajski, M. Meredith, and A. Takach, "An introduction to high-level synthesis," *IEEE Design & Test of Computers*, vol. 26, no. 4, pp. 8–17, Jul./Aug. 2009.

[8] J. Cong, B. Liu, S. Neuendorffer, J. Noguera, K. Vissers, and Z. Zhang, "High-level synthesis for FPGAs: From prototyping to deployment," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 30, no. 4, pp. 473–491, Apr. 2011.

[9] R. Nane *et al.*, "A survey and evaluation of FPGA high-level synthesis tools," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 35, no. 10, pp. 1591–1604, Oct. 2016.